

Autômatos finitos não determinísticos

1. Desenhe o diagrama de estados e descreva a linguagem aceita por cada um dos seguintes autômatos finitos não determinísticos. Em cada caso o estado inicial é q_1 .

(a) $F_1 = \{q_4\}$ e a função de transição é dada por:

δ_1	a	b	c
q_1	$\{q_1, q_2, q_3\}$	$\emptyset$	$\emptyset$
q_2	$\emptyset$	$\{q_4\}$	$\emptyset$
q_3	$\emptyset$	$\emptyset$	$\{q_4\}$
q_4	$\emptyset$	$\emptyset$	$\emptyset$

(b) $\delta_2 = \delta_1$ e $F_2 = \{q_1, q_2, q_3\}$.

(c) $F_3 = \{q_2\}$ e a função de transição é dada por:

δ_3	a	b
q_1	$\{q_2\}$	$\emptyset$
q_2	$\emptyset$	$\{q_1, q_3\}$
q_3	$\{q_1, q_3\}$	$\emptyset$

2. Determine, usando o algoritmo descrito em aula, autômatos finitos não determinísticos que aceitem as linguagens cujas expressões regulares são dadas abaixo:

- (a) $(10 \cup 001 \cup 010)^*$;
 (b) $(1 \cup 0)^*00101$;
 (c) $((0 \cdot 0) \cup (0 \cdot 0 \cdot 0))^*$.

3. Converta cada um dos autômatos finitos não determinísticos obtidos no exercício anterior em um autômato finito determinístico.
4. Seja A um autômato finito determinístico com um único estado final. Considere o autômato finito não determinístico A' obtido invertendo os papéis

dos estados inicial e final e invertendo também a direção de cada seta no diagrama de estado. Descreva $L(A')$ em termos de $L(A)$.

5. Seja $\Sigma = \{0, 1\}$. Seja $L_1 \subset \Sigma^*$ a linguagem que consiste das palavras onde há pelo menos duas ocorrências de 0, e $L_2 \subset \Sigma^*$ a linguagem que consiste das palavras onde há pelo menos uma ocorrência de 1. Para cada uma das linguagens L abaixo dê uma expressão regular para L e use os algoritmos descritos em aula para criar um autômato finito não determinístico que aceita L .
 - (a) $L = L_1 \cup L_2$;
 - (b) $L = \Sigma^* \setminus L_1$;
 - (c) $L = L_1 \cap L_2$.
6. Sejam L e L' linguagens regulares. Mostre que $L \setminus L'$ é regular
7. Sejam L e L' linguagens tais que L é regular, $L \cup L'$ é regular e $L \cap L' = \emptyset$. Mostre que L' é regular.
8. Sejam M e M' autômatos finitos não determinísticos em um alfabeto Σ . Neste exercício discutimos uma maneira de definir um autômato finito não determinístico N que aceita a concatenação $L(M) \cdot L(M')$ diferente da que foi vista em aula. Seja δ a função de transição de M . Para construir o grafo de N a partir dos grafos de M e M' procedemos da seguinte maneira. Toda vez que um estado q de M satisfaz

para algum $\sigma \in \Sigma$, o conjunto $\delta(q, \sigma)$ contém um estado final,

 acrescentamos uma transição de q para o estado inicial de M' , indexada por σ .
 - (a) Descreva detalhadamente todos os ingredientes de N . Quem são os estados finais de N ?
 - (b) Mostre que se $\epsilon \notin L(M)$ então N aceita $L(M) \cdot L(M')$.
 - (c) Se $\epsilon \in L(M)$ então pode ser necessário acrescentar mais uma transição para que o autômato aceite $L(M) \cdot L(M')$. Que transição é esta?
9. Seja M um autômato finito determinístico no alfabeto Σ , com estado inicial q_1 , conjunto de estados Q e função de transição δ . Dizemos que M tem *reinício* se existem $q \in Q$ e $\sigma \in \Sigma$ tais que

$$\delta(q, \sigma) = q_1.$$

Em outras palavras, no grafo de M há uma seta que aponta para q_1 . Mostre como é possível construir, a partir de um autômato finito determinístico M qualquer, um autômato finito determinístico M' *sem* reinício tal que $L(M) = L(M')$.

10. Sejam A e A' autômatos finitos determinísticos com alfabeto Σ , conjunto de estados Q e Q' , estados iniciais q_1 e q'_1 , conjuntos de estados finais F e F' e funções de transição δ e δ' . Seja M o autômato finito determinístico com alfabeto Σ , conjunto de estados $Q_1 \times Q_2$, estado inicial (q_1, q'_1) , conjunto

de estados finais $F_1 \times F_2$ e função de transição dada por

$$\delta((q, q'), \sigma) = (\delta_1(q, \sigma), \delta_2(q', \sigma)),$$

onde $q \in Q$, $q' \in Q'$ e $\sigma \in \Sigma$.

- (a) Mostre que $L(M) = L(A) \cap L(A')$.
 - (b) Use (a) para dar uma outra demonstração de que a interseção de linguagens regulares é regular.
 - (c) Use esta construção para obter um autômato finito determinístico que aceita a linguagem L do exercício 6(c).
11. Seja L uma linguagem que é regular. Definimos o *posto* de L como sendo o *menor* inteiro positivo k para o qual existe um autômato finito determinístico A com k estados e tal que $L = L(A)$.
- (a) Se L_1 e L_2 são linguagens regulares cujos postos são k_1 e k_2 respectivamente, determine m (em termos de k_1 e k_2) de modo que o posto de $L_1 \cdot L_2$ seja menor ou igual a m .
 - (b) Considere a afirmação: se $L_1 \subseteq L_2$ são linguagens regulares então o posto de L_1 tem que ser menor ou igual que o posto de L_2 . Esta afirmação é verdadeira ou falsa? Justifique sua resposta com cuidado!

Operações com autômatos finitos

No capítulo 3 vimos como obter a expressão regular da linguagem aceita por um autômato finito. Agora, começamos a preparar o terreno para poder resolver o problema inverso; isto é, dada uma expressão regular r em um alfabeto Σ , como construir um autômato finito $\mathcal{M}$ que aceita $L(r)$? Nossa estratégia consistirá em usar r como uma receita recursiva para construir $\mathcal{M}$. Contudo, r é construída, a partir dos símbolos de Σ , por aplicações repetidas das operações de união, concatenação e estrela. Assim, para poder usar r como uma receita para construir $\mathcal{M}$ precisamos antes solucionar o seguinte problema:

dados autômatos finitos $\mathcal{M}$ e $\mathcal{M}'$ em um alfabeto Σ construir autômatos finitos que aceitem $L(\mathcal{M}) \cup L(\mathcal{M}')$, $L(\mathcal{M}) \cdot L(\mathcal{M}')$ e $L(\mathcal{M})^*$.

É exatamente isto que faremos neste capítulo.

1. União

Suponhamos que $\mathcal{M}$ e $\mathcal{M}'$ são autômatos finitos não determinísticos em um mesmo alfabeto Σ . Mais precisamente, digamos que

$$(1.1) \quad \mathcal{M} = (\Sigma, q_1, Q, F, \delta) \text{ e } \mathcal{M}' = (\Sigma, q'_1, Q', F', \delta').$$

Assumiremos, também, que $Q \cap Q' = \emptyset$. Observe que esta hipótese não representa nenhuma restrição expressiva, já que para alcançá-la basta renomear os estados de $\mathcal{M}'$, no caso de serem denotados pelo mesmo nome que os estados de $\mathcal{M}$.

Um outro detalhe que vale a pena mencionar é que, tanto nesta construção, como na da concatenação, estamos supondo que *ambos os autômatos estão definidos para um mesmo alfabeto*. Novamente, esta restrição é apenas aparente já que os autômatos não precisam ser determinísticos. De fato, podemos aumentar o alfabeto de entrada de um autômato não determinístico o quanto quisermos, sem alterar o seu comportamento. Para isso basta decretar que as

transições pelos novos estados são todas vazias. No caso da união, isto significa que, se $\mathcal{M}$ e $\mathcal{M}'$ tiverem alfabetos de entrada Σ e Σ' diferentes, então podemos considerar ambos como autômatos no alfabeto $\Sigma \cup \Sigma'$. Para mais detalhes veja o exercício 1.

Vejamos como deve ser o comportamento de um autômato finito $\mathcal{M}_u$ para que aceite $L(\mathcal{M}) \cup L(\mathcal{M}')$. O autômato $\mathcal{M}_u$ aceitará uma palavra $w \in \Sigma^*$ somente quando w for aceita por $\mathcal{M}$ ou por $\mathcal{M}'$. Mas, para descobrir isto, $\mathcal{M}_u$ deve ser capaz de simular estes dois autômatos. Como estamos partindo do princípio de que $\mathcal{M}_u$ é não determinístico, podemos deixá-lo escolher qual dos dois autômatos vai querer simular em uma dada computação. Portanto, ao receber uma palavra w , o autômato $\mathcal{M}_u$:

- escolhe se vai simular $\mathcal{M}$ ou $\mathcal{M}'$;
- executa a simulação escolhida e aceita w apenas se for aceita pelo autômato cuja simulação está sendo executada.

Uma maneira de realizar isto em um autômato finito é criar um novo estado inicial q_0 cuja única função é realizar a escolha de qual dos dois autômatos será simulado. Mais uma vez, como $\mathcal{M}_u$ não é determinístico, podemos deixá-lo decidir, por si próprio, qual o autômato que será simulado.

Ainda assim, resta um problema. De fato, todos os símbolos de w serão necessários para que as simulações de $\mathcal{M}$ e $\mathcal{M}'$ funcionem corretamente. Contudo, nossos autômatos precisam consumir símbolos para efetuar suas transições. Digamos, por exemplo, que ao receber w no estado q_0 o autômato $\mathcal{M}_u$ escolhe simular $\mathcal{M}$ com entrada w . Entretanto, para poder simular $\mathcal{M}$ em w , $\mathcal{M}_u$ precisa deixar q_0 e chegar ao estado inicial q_1 de $\mathcal{M}$ ainda tendo w por entrada, o que não é possível. Resolvemos este problema fazendo com que q_0 comporte-se, simultaneamente, como q_1 ou como q'_1 , dependendo de qual autômato $\mathcal{M}_u$ escolheu simular. Mais precisamente, se $\mathcal{M}_u$ escolheu simular $\mathcal{M}$, então q_0 realiza a transição a partir do primeiro símbolo de w como se fosse uma transição de $\mathcal{M}$ a partir de q_1 por este mesmo símbolo.

Para poder formalizar isto, digamos que $w = \sigma u$, onde $\sigma \in \Sigma$ e $u \in \Sigma^*$. Neste caso, se $\mathcal{M}_u$ escolheu simular $\mathcal{M}$, devemos ter que

$$(q_0, w) \vdash (p, u) \text{ se, e somente se, } p \in \delta(q_1, \sigma).$$

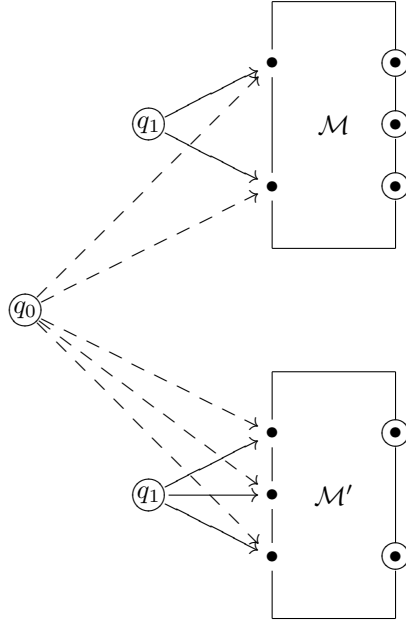
Podemos dissecar o comportamento de $\mathcal{M}_u$ a partir de q_0 como duas escolhas sucessivas:

- (1) primeiro $\mathcal{M}_u$ decide se vai simular $\mathcal{M}$ ou $\mathcal{M}'$;
- (2) a seguir, $\mathcal{M}_u$ escolhe qual a transição que vai ser executada por σ , a partir de q_1 ou q'_1 —dependendo da escolha feita em (1).

Denotando por δ_u a função de transição de $\mathcal{M}_u$, podemos resumir isto escrevendo

$$\delta_u(q_0, \sigma) = \delta(q_1, \sigma) \cup \delta'(q'_1, \sigma).$$

O comportamento geral de $\mathcal{M}_u$ pode ser ilustrado em uma figura, como segue. As transições que foram acrescentadas como parte da construção de $\mathcal{M}_u$ aparecem tracejadas.



Como a figura sugere, os estados de $\mathcal{M}_u$ são os mesmos de $\mathcal{M}$ e $\mathcal{M}'$, exceto por q_0 . Quanto aos estados finais, precisamos ser mais cuidadosos. Em primeiro lugar, uma vez tendo passado por q_0 , o autômato $\mathcal{M}_u$ meramente simula $\mathcal{M}$ ou $\mathcal{M}'$. Portanto, a descrição dos estados finais de $\mathcal{M}_u$ só apresenta algum problema se $q_1 \in F$ ou $q'_1 \in F'$. Por exemplo, se q_1 for final, então $\mathcal{M}$ e, portanto, $\mathcal{M}_u$, deverá aceitar ϵ . Mas isto só pode ocorrer se q_0 for final. Portanto, q_0 deverá ser declarado como final sempre que q_1 ou q'_1 forem finais.

Vamos formalizar a construção, listando os vários elementos de $\mathcal{M}_u$ um a um:

Alfabeto: Σ ;

Estados: $\{q_0\} \cup Q \cup Q'$;

Estado inicial: q_0 ;

Estados finais:

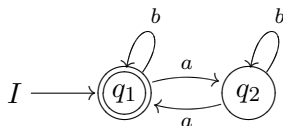
$$\{q_0\} \cup F \cup F' \text{ se } q_1 \in F \text{ ou } q'_1 \in F'$$

$$F \cup F' \text{ se } q_1 \notin F \text{ e } q'_1 \notin F'$$

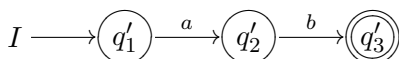
Função de transição:

$$\delta_u(q, \sigma) = \begin{cases} \delta(q_1, \sigma) \cup \delta'(q'_1, \sigma) & \text{se } q = q_0 \\ \delta(q, \sigma) & \text{se } q \in Q \\ \delta'(q, \sigma) & \text{se } q \in Q' \end{cases}$$

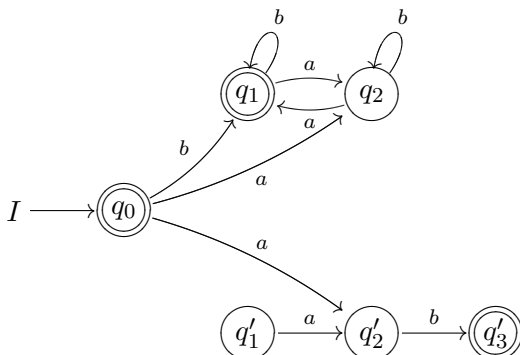
Vejamos como a construção se comporta em um exemplo. Considere o autômato finito determinístico $\mathcal{M}_{\text{par}}$, com grafo



e o autômato finito não determinístico $\mathcal{M}_{ab}$ cujo grafo é



Já sabemos que $\mathcal{M}$ aceita a linguagem formada pelas palavras no alfabeto $\{a, b\}$ que têm uma quantidade par de as , e que $L(\mathcal{M}') = \{ab\}$. Aplicando a construção discutida acima para construir um autômato $\mathcal{M}_u$ que aceita $L(\mathcal{M}) \cup L(\mathcal{M}')$, obtemos



Há dois detalhes importantes a serem observados neste exemplo. O primeiro é que, como há uma seta de q_1 nele próprio, indexada por a , você poderia esperar encontrar em $\mathcal{M}_u$ uma seta de q_0 nele próprio com o mesmo índice. No entanto, não é isto que acontece. De fato, denotando por δ a função de transição de $\mathcal{M}$ temos, pela definição de $\mathcal{M}_u$, que

$$\delta(q_1, b) = q_1 \text{ implica } q_1 \in \delta_u(q_0, b).$$

Em outras palavras, a transição por b em $\mathcal{M}$, que leva q_1 nele próprio, dá lugar a uma transição por b de q_0 em q_1 .

O segundo detalhe diz respeito à eliminação de estados redundantes. À primeira vista, como q_0 passa a desempenhar o papel de estado inicial, em substituição a q_1 e q'_1 , então estes últimos estados deveriam ter-se tornado redundantes. Entretanto, embora q'_1 seja, de fato, redundante, o mesmo não ocorre com q_1 . Por exemplo, o autômato $\mathcal{M}$ retorna a q_1

durante a computação

$$(q_1, a^2) \vdash (q_2, a) \vdash (q_1, \epsilon).$$

Simulando a mesma computação em $\mathcal{M}_u$, teremos

$$(q_0, a^2) \vdash (q_2, a) \vdash (q_1, \epsilon).$$

Note que embora $\mathcal{M}_u$ parta de q_0 , na terceira etapa da computação ele alcança q_1 , exatamente como $\mathcal{M}$. Assim, q_0 substitui q_1 *apenas quando este último está desempenhando o papel de estado inicial*.

Para esclarecer, de forma definitiva, todos os detalhes da construção de $\mathcal{M}_u$, provaremos formalmente que

$$L(\mathcal{M}_u) = L(\mathcal{M}) \cup L(\mathcal{M}').$$

Faremos isto no caso geral, admitindo que $\mathcal{M}$ e $\mathcal{M}'$ são os autômatos não determinísticos definidos em (1.1). Nossa demonstração consiste em provar que $w \in \Sigma^*$ é aceita por $\mathcal{M}_u$ se, e somente se, for aceita por $\mathcal{M}$ ou $\mathcal{M}'$. Se $w = \epsilon$ não há muito o que dizer, por isso deixamos este caso aos seus cuidados. De agora em diante estaremos assumindo que $w \neq \epsilon$. Com isso, podemos escrever w na forma

$$w = \sigma u, \text{ onde } \sigma \in \Sigma \text{ e } u \in \Sigma^*.$$

Antes de prosseguir, convém enunciar claramente as propriedades de $\mathcal{M}_u$ que serão utilizadas na demonstração. Em primeiro lugar, os estados e transições de $\mathcal{M}$ e de $\mathcal{M}'$ correspondem um a um, aos estados e transições de $\mathcal{M}_u$, exceto por q_0 e suas transições. Assim,

- toda computação em $\mathcal{M}$ pode ser reproduzida literalmente (quer dizer, com os mesmos estados e transições) como uma computação de $\mathcal{M}_u$;
- reciprocamente, qualquer computação de $\mathcal{M}_u$ a partir de um estado de $\mathcal{M}$ pode ser reproduzida literalmente em $\mathcal{M}$.

Naturalmente, as mesmas afirmações valem se substituirmos $\mathcal{M}$ por $\mathcal{M}'$. Em segundo lugar,

$$(1.2) \quad \text{se } p \in \delta(q_1, \sigma) \text{ então } p \in \delta_u(q_0, \sigma).$$

Digamos, primeiramente, que $w \in L(\mathcal{M})$. Temos, então, uma computação

$$(q_1, \sigma u) \vdash (p, u) \vdash^* (f, \epsilon),$$

onde $p \in Q$ e $f \in F$. Mas, por (1.2),

$$\text{se } (q_1, \sigma u) \vdash (p, u) \text{ em } \mathcal{M}, \text{ então } (q_1, \sigma u) \vdash (p, u) \text{ em } \mathcal{M}_u.$$

Por outro lado, da primeira propriedade acima, a computação $(p, u) \vdash^* (f, \epsilon)$ em $\mathcal{M}$ pode ser reproduzida, passo a passo, e com os mesmos

estados e transições, como uma computação de $\mathcal{M}_u$. Obtemos, assim, a computação

$$(q_0, \sigma u) \vdash (p, u) \vdash^* (f, \epsilon),$$

em $\mathcal{M}_u$. Como f é final em $\mathcal{M}$, ele também será final em $\mathcal{M}_u$, de modo que toda palavra aceita por $\mathcal{M}$ será aceita por $\mathcal{M}_u$; isto é $L(\mathcal{M}) \subseteq L(\mathcal{M}_u)$. Um argumento análogo mostra que $L(\mathcal{M}') \subseteq L(\mathcal{M}_u)$.

Esta primeira parte do argumento pressupõe que todos os estados não redundantes de $\mathcal{M}$ e $\mathcal{M}'$ sejam também estados de $\mathcal{M}_u$. Mas há uma exceção a esta afirmação. Digamos, por exemplo, que q_1 seja inacessível a partir de qualquer estado de $\mathcal{M}$, *inclusive dele próprio*. Neste caso, q_1 vai se tornar redundante em $\mathcal{M}_u$. Isto acontece porque uma computação só pode passar por q_1 uma vez e, mesmo assim, somente se começar deste estado, ao passo que as computações de $\mathcal{M}_u$ começam de q_0 . Este fenômeno ocorre, por exemplo, com o estado q'_1 no exemplo descrito acima.

Suponha, agora, que

$$w = \sigma u \in L(\mathcal{M}_u).$$

Teremos, então, uma computação em $\mathcal{M}_u$ da forma

$$(1.3) \quad (q_0, \sigma u) \vdash (p, u) \vdash^* (f, \epsilon),$$

onde f é um estado final de $\mathcal{M}_u$. Pela definição das transições em $\mathcal{M}_u$ devemos ter que $p \in Q$ ou $p \in Q'$. Suporemos, sem perda de generalidade, que $p \in Q$. Contudo, qualquer transição em $\mathcal{M}_u$ a partir de um estado de $\mathcal{M}$ também é uma transição de $\mathcal{M}$. Como $p \in Q$, isto significa que

$$(p, u) \vdash^* (f, \epsilon)$$

terá que corresponder, passo a passo, a uma computação em $\mathcal{M}$. Portanto, usando (1.2), obtemos uma computação

$$(q_1, \sigma u) \vdash (p, u) \vdash^* (f, \epsilon),$$

em $\mathcal{M}$. Como f é um estado final de $\mathcal{M}_u$ que pertence a Q , concluímos que $f \in F$. Logo, w é aceita por $\mathcal{M}$.

Observe que a segunda parte da demonstração está ancorada no fato de nenhuma transição de $\mathcal{M}_u$ poder chegar em q_0 . Se isto pudesse acontecer, $\mathcal{M}_u$ poderia executar parte da computação em $\mathcal{M}$, retornar a q_0 , e continuar com uma computação de $\mathcal{M}'$. Entretanto, o argumento acima só funciona porque, uma vez que $\mathcal{M}_u$ tenha alcançado um estado de $\mathcal{M}$, ele fica *preso* neste último autômato, e assim é obrigado a se comportar como $\mathcal{M}$.

2. Concatenação

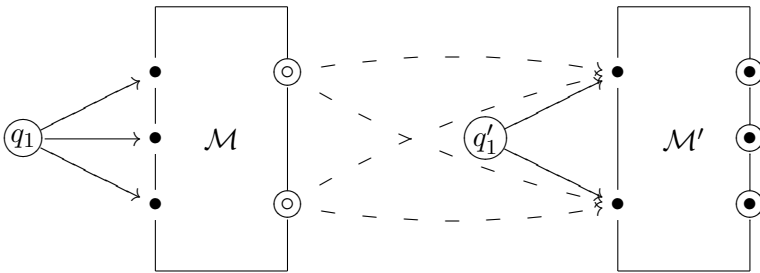
A segunda operação de linguagens que precisamos transcrever para os autômatos é a concatenação. Suponhamos, mais uma vez, que temos dois autômatos $\mathcal{M}$ e $\mathcal{M}'$ no mesmo alfabeto Σ , cujos elementos são

$$\mathcal{M} = (\Sigma, q_1, Q, F, \delta) \text{ e } \mathcal{M}' = (\Sigma, q'_1, Q', F', \delta').$$

Continuaremos assumindo que $Q \cap Q' = \emptyset$.

Nosso objetivo é construir um autômato $\mathcal{M}_c$ que aceite a concatenação $L(\mathcal{M}) \cdot L(\mathcal{M}')$. Assim, dada uma palavra $w \in \Sigma^*$, o autômato $\mathcal{M}_c$ precisa verificar se existe um prefixo de w que é aceito por $\mathcal{M}$ e que é seguido de um sufixo aceito por $\mathcal{M}'$. Naturalmente isto sugere conectar os autômatos $\mathcal{M}$ e $\mathcal{M}'$ ‘em série’; quer dizer, um depois do outro. Além disso, para que o prefixo esteja em $L(\mathcal{M})$ o último estado de $\mathcal{M}$ alcançado em uma computação por w deve ser final. Finalmente, é mais fácil construir um modelo não determinístico de $\mathcal{M}_c$, já que não temos como saber exatamente onde acaba o prefixo de w que pertence a $L(\mathcal{M})$.

Nossa experiência com a união deixa claro que não adianta conectar os autômatos através de transições de um estado final de $\mathcal{M}$ para o estado inicial de $\mathcal{M}'$. Isto só seria possível se nossos autômatos pudessem efetuar transições sem consumir nenhum símbolo da entrada. Contornamos o problema, como fizemos no caso da união, construindo as transições diretamente dos estados finais de $\mathcal{M}$ para os sucessores do estado inicial de $\mathcal{M}'$. Podemos ilustrar a construção em uma figura, como segue.



As setas correspondentes às transições entre os autômatos $\mathcal{M}$ e $\mathcal{M}'$ foram tracejadas para dar maior clareza à figura. Observe também que os estados que seriam finais em $\mathcal{M}$ aparecem com a bolinha central vazada ($\circ$ em vez de $\bullet$). Fizemos isto porque nem sempre um estado final de $\mathcal{M}$ continuará final em $\mathcal{M}_c$. Na verdade, se todo estado em F for final em $\mathcal{M}_c$ então toda computação em $\mathcal{M}$ que acabar em F será

aceita por $\mathcal{M}_c$. Quer dizer, toda palavra em $L(\mathcal{M})$ também pertence a $L(\mathcal{M}_c) = L(\mathcal{M}) \cdot L(\mathcal{M}')$. Entretanto, isto só pode acontecer se $\epsilon \in L(\mathcal{M}')$; ou, o que é equivalente, se q'_1 é estado final de $\mathcal{M}'$. Caso contrário, apenas os estados finais de $\mathcal{M}'$ serão finais em $\mathcal{M}$.

Formalizaremos a construção, listando os vários elementos de $\mathcal{M}_c$ um a um:

Alfabeto: Σ ;

Estados: $Q \cup Q'$;

Estado inicial: q_1 ;

Estados finais:

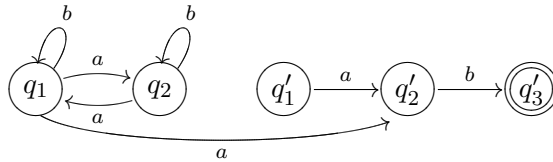
$$F \cup F' \text{ se } q'_1 \in F'$$

$$F' \text{ se } q'_1 \notin F'$$

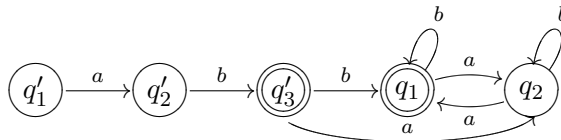
Função de transição:

$$\delta_c(q, \sigma) = \begin{cases} \delta(q, \sigma) & \text{se } q \in Q \setminus F \\ \delta(q, \sigma) \cup \delta'(q'_1, \sigma) & \text{se } q \in F \\ \delta'(q, \sigma) & \text{se } q \in Q' \end{cases}$$

A demonstração de que esta construção funciona corretamente é bastante simples e segue a linha da demonstração para a união dada na seção anterior, por isso vamos omiti-la. Vejamos como a construção se comporta quando é aplicada aos autômatos utilizados no exemplo da seção 1. Começaremos concatenando $\mathcal{M}_{\text{par}}$ com $\mathcal{M}_{ab}$:



Observe que o estado q_1 deixou de ser final nesta concatenação, porque q'_1 não é estado final de $\mathcal{M}_{ab}$. Por outro lado, se concatenarmos $\mathcal{M}_{ab}$ com $\mathcal{M}_{\text{par}}$, então q'_3 continuará sendo final, já que q_1 é final em $\mathcal{M}_{\text{par}}$. O grafo do autômato resultante desta concatenação é o seguinte:

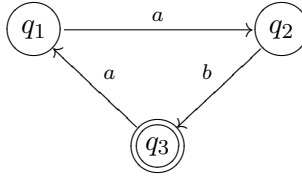


Neste caso temos duas transições a partir do estado final de $\mathcal{M}_{ab}$, porque q_1 tem como sucessores q_2 e o próprio q_1 .

3. Estrela

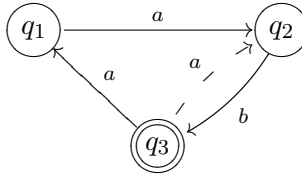
Tendo mostrado como construir um autômato para a concatenação, não parece tão difícil lidar com a estrela. Afinal, se L é uma linguagem, então L^* é obtida concatenando L com ela própria mais e mais vezes; isto é, calculando a união das L^j , para todo $j \geq 0$. Portanto, uma maneira de realizar um autômato $\mathcal{M}$, que aceite L , consistiria em conectar a saída de $\mathcal{M}$ com sua entrada. Com isso a concatenação passaria infinitas vezes pelo próprio $\mathcal{M}$, e teríamos um novo autômato $\mathcal{M}^*$ que aceita L^* . Vamos testar esta idéia em um exemplo e ver o que acontece.

Considere o autômato finito $\mathcal{N}$ no alfabeto $\{a, b\}$, cujo grafo é dado por



A linguagem aceita por $\mathcal{N}$ é denotada pela expressão regular $(aba)^*ab$, como é fácil de ver.

Para concatenar $\mathcal{N}$ com ele próprio precisamos criar transições entre seus estados finais e os sucessores do estado inicial. Neste exemplo, há apenas o estado final q_3 , e q_1 tem apenas um sucessor; a saber, o estado q_2 que é acessível a partir de q_1 por a . Por isso precisamos criar uma nova transição de q_3 para q_2 por a , obtendo o seguinte autômato

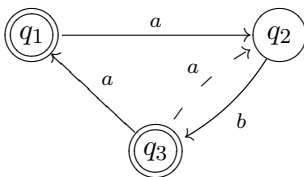


A transição acrescentada foi desenhada tracejada para que possa ser mais facilmente identificada.

Infelizmente, não pode ser verdade que este autômato aceite $L(\mathcal{N})^*$. Afinal, ϵ pertence a $L(\mathcal{N})^*$ e não é aceito pelo autômato que acabamos de construir. A essa altura, sua reação pode ser:

“Não seja por isso! Para resolver este problema basta marcar q_1 como sendo estado final do novo autômato.”

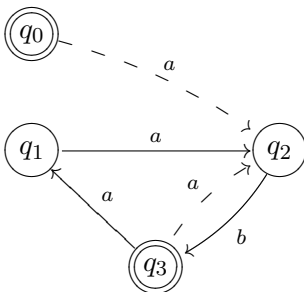
Vamos fazer isto e ver o que acontece. O grafo do autômato passaria a ser o seguinte:



Contudo, este autômato aceita a palavra aba que não pertence a

$$L(\mathcal{N})^* = ((aba)^*ab)^*.$$

O problema não está em nossa idéia original de concatenar um autômato com ele próprio, mas sim na emenda que adotamos para resolver o problema de fazer o novo autômato aceitar ϵ . Ao declarar q_1 como sendo estado final, não levamos em conta que o autômato admite transições que retornam ao estado q_1 . Isto fez com que outras palavras, além de ϵ , passassem a ser aceitas pelo novo autômato. Mas não é isso que queremos. A construção original funcionava perfeitamente, exceto porque ϵ não era aceita. Uma maneira simples de contornar o problema é inspirar-se na união, e criar um novo estado q_0 para fazer o papel de estado inicial. Este estado vai comportar-se como q_1 , e também será final. Entretanto, como no caso da união, as transições por q_0 são exatamente as mesmas que as transições por q_1 . Em particular, nenhuma transição do novo autômato aponta para q_0 . Isto garante que a introdução de q_0 leva à aceitação de apenas uma nova palavra, que é ϵ . Esboçamos abaixo o grafo do autômato resultante desta construção, no exemplo que vimos considerando.

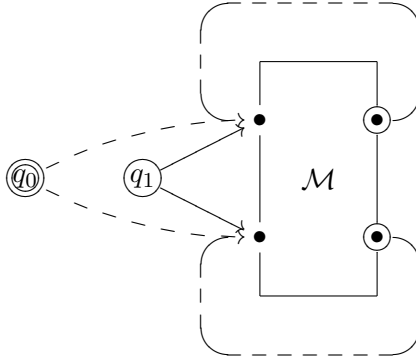


Em geral, quando é dado um autômato

$$\mathcal{M} = (\Sigma, q_1, Q, F, \delta)$$

podemos construir um autômato $\mathcal{M}^*$ que aceita $L(\mathcal{M})^*$ segundo o modelo estabelecido no exemplo anterior. Como no caso da união e da concatenação, podemos ilustrar o autômato $\mathcal{M}^*$ em uma figura nas quais as

novas transições aparecem tracejadas.



Formalizaremos a construção, listando os vários elementos de $\mathcal{M}^*$, como segue:

Alfabeto: Σ ;

Estados: $\{q_0\} \cup Q$;

Estado inicial: q_0 ;

Estados finais: $F \cup \{q_0\}$

Função de transição:

$$\delta^*(q, \sigma) = \begin{cases} \delta(q_1, \sigma) & \text{se } q = q_0 \\ \delta(q, \sigma) & \text{se } q \in Q \setminus F \\ \delta(q, \sigma) \cup \delta(q_1, \sigma) & \text{se } q \in F \end{cases}$$

Encerramos a seção com a demonstração de que esta construção funciona corretamente. Em primeiro lugar, como ϵ é aceita por construção, podemos limitar nossa discussão a uma palavra $w \neq \epsilon$ que pertença a Σ^* .

Como em nossa discussão do autômato que aceita a união de linguagens, isolaremos duas propriedades da definição de $\mathcal{M}^*$ que serão utilizadas na demonstração. Em primeiro lugar, se $(q_1, \sigma) \vdash (p, \epsilon)$ em $\mathcal{M}$, então existirão transições em $\mathcal{M}^*$ da forma

$$(q_0, \sigma) \vdash (p, \epsilon),$$

e, também, da forma

$$(f, \sigma) \vdash (p, \epsilon) \text{ para cada estado final } f \text{ de } \mathcal{M}.$$

Que q_0 comporta-se como se fosse q_1 é óbvio da maneira como suas transições foram definidas. Já a segunda parte da afirmação segue do fato de que também fizemos cada estado final de $\mathcal{M}^*$ comportar-se como se fosse q_1 ao concatenar $\mathcal{M}$ com ele próprio. Outro ponto importante é que toda computação em $\mathcal{M}$ pode ser simulada por $\mathcal{M}^*$ com os mesmos

estados e transições, já que $\mathcal{M}$ foi inteiramente preservado “dentro” de $\mathcal{M}^*$.

Seja $L = L(\mathcal{M})$. Como sempre, temos que provar que toda palavra que pertence a L^* é aceita por $\mathcal{M}^*$, e vice-versa. Começaremos mostrando $L^* \subseteq L(\mathcal{M}^*)$. Digamos que $w \in L^*$. Neste caso, podemos escrever w na forma

$$w = w_1 w_2 \cdots w_k \text{ onde } w_1, w_2, \dots, w_k \in L.$$

Digamos que $w_j = \sigma_j u_j$, com $\sigma_j \in \Sigma$ e $u_j \in \Sigma^*$. Como cada w_j pertence a L , tem que existir uma computação

$$(q_1, w_j) \vdash (p_j, u_j) \vdash^* (f_j, \epsilon),$$

onde $p_j \in Q$ e $f_j \in F$. Mas o estado inicial q_0 de $\mathcal{M}^*$ simula o comportamento de q_1 , e $\mathcal{M}^*$ é capaz de simular qualquer computação em $\mathcal{M}$. Portanto, podemos construir uma computação em $\mathcal{M}^*$ da forma:

$$(q_0, \sigma_1 u_1 w_2 \cdots w_k) \vdash (p_1, u_1 w_2 \cdots w_k) \vdash^* (f_1, w_2 \cdots w_k).$$

Por outro lado, também podemos construir computações da forma

$$(f_j, w_{j+1} \cdots w_k) = (f_j, \sigma_{j+1} u_{j+1} \cdots w_k) \vdash (p_{j+1}, u_{j+1} \cdots w_k) \vdash^* (f_{j+1}, w_{j+2} \cdots w_k),$$

para todo $1 \leq j < k$. Emendando o início de cada uma dessas computações ao final da outra, obtemos

$$(q_0, w_1 w_2 \cdots w_k) \vdash^* (f_1, w_2 \cdots w_k) \vdash^* (f_2, w_3 \cdots w_k) \cdots \vdash^* (f_{k-1}, w_k) \vdash^* (f_k, \epsilon),$$

provando, assim, que $\mathcal{M}^*$ aceita w .

Passamos, agora, a mostrar que $L(\mathcal{M}^*) \subseteq L^*$. Para facilitar a discussão, distinguiremos dois tipos de transição em $\mathcal{M}^*$:

Primeiro tipo: as transições a partir de q_0 , além de todas aquelas que já eram transições de $\mathcal{M}$;

Segundo tipo: transições que $\mathcal{M}^*$ faz a partir dos estados finais de $\mathcal{M}$, como se fossem q_1 .

Suponha, agora, que w é aceita por $\mathcal{M}^*$. Neste caso existe uma computação

$$(q_0, w) \vdash^* (f, \epsilon),$$

em $\mathcal{M}^*$. Percorrendo, agora, esta computação, vamos dividi-la em partes de modo que a primeira transição de cada segmento, a partir do segundo, seja uma transição do segundo tipo. Se w_j for a subpalavra de

w consumida ao longo do j -ésimo segmento, podemos escrever os segmentos na forma

$$\begin{aligned} (q_0, w) &\vdash^* (f_1, w_2 \cdots w_k) \\ (f_1, w_2 \cdots w_k) &\vdash^* (f_2, w_3 \cdots w_k) \\ &\vdots \\ (f_{k-1}, w_k) &\vdash^* (f_k, \epsilon), \end{aligned}$$

onde $f_1, \dots, f_k \in F$. Além disso, se $w_j = \sigma_j u_j$, com $\sigma_j \in \Sigma$ e $u_j \in \Sigma^*$, então a transição do segundo tipo pode ser isolada em cada segmento

$$\begin{aligned} (f_j, w_j w_{j+1} \cdots w_k) &= (f_j, \sigma_j u_j w_{j+1} \cdots w_k) \vdash (p_j, u_j w_{j+1} \cdots w_k) \\ &\vdash^* (f_{j+1}, w_{j+1} \cdots w_k), \end{aligned}$$

para todo $1 \leq j < k$, onde $p_j \in Q$. Temos, assim, computações da forma

$$(f_j, w_j) = (f_j, \sigma_j u_j) \vdash (p_j, u_j) \vdash^* (f_{j+1}, \epsilon),$$

em $\mathcal{M}^*$, que por sua vez dão lugar a computações da forma

$$(q_1, w_j) = (f_j, \sigma_j u_j) \vdash (p_j, u_j) \vdash^* (f_{j+1}, \epsilon),$$

em $\mathcal{M}$. Portanto, $w_2, \dots, w_k \in L$. Um argumento semelhante mostra que $w_1 \in L$, e nos permite concluir que

$$w = w_1 w_2 \cdots w_k \in L^*,$$

como nos faltava mostrar.

4. Exercícios

1. Sejam $\Sigma \subset \Sigma'$ dois alfabetos. Suponha que $\mathcal{M}$ é um autômato finito não determinístico no alfabeto Σ . Construa formalmente um autômato finito não determinístico $\mathcal{M}'$ no alfabeto Σ' , de modo que $L(\mathcal{M}) = L(\mathcal{M}')$. Prove que sua construção funciona corretamente.
SUGESTÃO: Defina todas as transições de $\mathcal{M}'$ por símbolos de $\Sigma' \setminus \Sigma$ como sendo vazias.
2. Seja $\Sigma = \{0, 1\}$. Suponha que $L_1 \subset \Sigma^*$ é a linguagem que consiste das palavras onde há pelo menos duas ocorrências de 0, e que $L_2 \subset \Sigma^*$ é a linguagem que consiste das palavras onde há pelo menos uma ocorrência de 1.
 - (a) Construa autômatos finitos não determinísticos que aceitem L_1 e L_2 .
 - (b) Use os algoritmos desta seção para criar autômatos finitos não determinísticos que aceitem $L_1 \cup L_2$, $L_1 \cdot L_2$, L_1^* e L_2^* .

3. Sejam M e M' autômatos finitos não determinísticos em um alfabeto Σ . Neste exercício discutimos uma maneira de definir um autômato finito não determinístico N , que aceita a concatenação $L(M) \cdot L(M')$, e que é diferente do que foi vista na seção 2. Seja δ a função de transição de M . Para construir o grafo de N a partir dos grafos de M e M' procedemos da seguinte maneira. Toda vez que um estado q de M satisfaz

para algum $\sigma \in \Sigma$, o conjunto $\delta(q, \sigma)$ contém um estado final,

acrescentamos uma transição de q para o estado inicial de M' , indexada por σ .

- Descreva detalhadamente todos os elementos de N . Quem são os estados finais de N ?
 - Mostre que se $\epsilon \notin L(M)$ então N aceita $L(M) \cdot L(M')$.
 - Se $\epsilon \in L(M)$ então pode ser necessário acrescentar mais uma transição para que o autômato aceite $L(M) \cdot L(M')$. Que transição é esta?
4. Seja M um autômato finito determinístico no alfabeto Σ , com estado inicial q_1 , conjunto de estados Q e função de transição δ . Dizemos que M tem *reinício* se existem $q \in Q$ e $\sigma \in \Sigma$ tais que

$$\delta(q, \sigma) = q_1.$$

Em outras palavras, no grafo de M há uma seta que aponta para q_1 . Mostre como é possível construir, a partir de um autômato finito determinístico M qualquer, um autômato finito determinístico M' *sem* reinício tal que $L(M) = L(M')$.

5. Sejam M e M' autômatos finitos *determinísticos* no alfabeto Σ cujos elementos são $(\Sigma, Q, q_1, F, \delta)$ e $(\Sigma, Q', q'_1, F', \delta')$, respectivamente. Defina um novo autômato finito *determinístico* N construído a partir de M e M' como segue:

Alfabeto: Σ ;

Estados: pares (q, q') onde $q \in Q$ e $q' \in Q'$;

Estado inicial: (q_1, q'_1) ;

Estados finais: pares (q, q') onde $q \in F$ ou $q' \in F'$;

Transição: a função de transição é definida em um estado (q, q') de N e símbolo $\sigma \in \Sigma$ por

$$\hat{\delta}((q, q'), \sigma) = (p, p'),$$

onde $p = \delta(q, \sigma)$ e $p' = \delta'(q', \sigma)$.

Calcule $L(N)$ em função de $L(M)$ e $L(M')$ e prove que a sua resposta está correta.

Autômatos de expressões regulares

No capítulo ?? vimos como obter a expressão regular da linguagem aceita por um autômato finito. Neste capítulo, resolvemos o problema inverso; isto é, dada uma expressão regular r em um alfabeto Σ , construímos um autômato finito que aceita $L(r)$. Na verdade, o autômato que resultará de nossa construção não será determinístico. Mas isto não é um problema, já que sabemos como convertê-lo em um autômato determinístico usando a construção de subconjuntos.

1. Considerações gerais

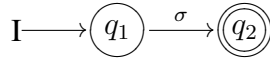
Seja r uma expressão regular r no alfabeto Σ . Nossa estratégia consistirá em utilizar a expressão regular r como uma receita para a construção de um autômato finito não determinístico $\mathcal{M}(r)$ que aceita $L(r)$. Entretanto, como vimos no capítulo ??, r é definida, de maneira recursiva, a partir dos símbolos de Σ , ϵ e $\emptyset$, por aplicação sucessiva das operações de união, concatenação e estrela. Por isso, efetuaremos a construção de $\mathcal{M}(r)$ passo à passo, a partir dos autômatos que aceitam os símbolos de Σ , ϵ e $\emptyset$.

Contudo, para que isto seja possível, precisamos antes de resolver alguns problemas. O primeiro, e mais simples, consiste em construir autômatos finitos que aceitem um símbolos de Σ , ou ϵ ou $\emptyset$. Estes autômatos funcionarão como os átomos da construção. Os problemas mais interessantes dizem respeito à construção de novos autômatos a partir de autômatos já existentes. Suponhamos, então, que $\mathcal{M}_1$ e $\mathcal{M}_2$ são autômatos finitos não determinísticos. Precisamos saber construir

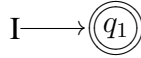
- (1) um autômato $\mathcal{M}_u$ que aceita $L(\mathcal{M}_1) \cup L(\mathcal{M}_2)$;
- (2) um autômato $\mathcal{M}_c$ que aceita $L(\mathcal{M}_1) \cdot L(\mathcal{M}_2)$; e
- (3) um autômato $\mathcal{M}_1^*$ que aceita $L(\mathcal{M}_1)^*$.

Se formos capazes de inventar maneiras de construir todos estes autômatos, então seremos capazes de reproduzir a montagem de r passo à passo no domínio dos autômatos finitos, o que nos levará a um autômato finito que aceita $L(r)$, como desejamos.

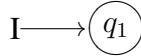
Começaremos descrevendo os átomos desta construção; isto é, os autômatos que aceitam símbolos de Σ , ϵ e $\emptyset$. Seja $\sigma \in \Sigma$. Um autômato simples que aceita σ é dado pelo grafo



O autômato que aceita ϵ é ainda mais simples



Para obter o que aceita $\emptyset$ basta não declarar nenhum estado como sendo final. A maneira mais simples de fazer isto é alterar o autômato acima, como segue.



Construídos os átomos, falta-nos determinar como podem ser conectados para obter estruturas maiores e mais complicadas. Passamos, assim, à solução dos problemas enunciados acima.

2. União

Suponhamos que $\mathcal{M}$ e $\mathcal{M}'$ são autômatos finitos não determinísticos em um mesmo alfabeto Σ . Mais precisamente, digamos que

$$\mathcal{M} = (\Sigma, q_1, Q, F, \delta) \text{ e } \mathcal{M}' = (\Sigma, q'_1, Q', F', \delta').$$

Suporemos, também, que $Q \cap Q' = \emptyset$. Isto não representa nenhuma restrição expressiva. Significa, no máximo, que pode ser necessário renomear os estados de $\mathcal{M}'$ caso sejam denotados pelo mesmo nome que os estados de $\mathcal{M}$.

Vejamos como deve ser o comportamento de um autômato finito $\mathcal{M}_u$ para que aceite $L(\mathcal{M}) \cup L(\mathcal{M}')$. Dada uma palavra $w \in \Sigma^*$ a $\mathcal{M}_u$ ele deve aceitá-la apenas se w for aceita por $\mathcal{M}$ ou por $\mathcal{M}'$. Mas, para descobrir isto, $\mathcal{M}_u$ deve ser capaz de simular estes dois autômatos. Como estamos partindo do princípio que $\mathcal{M}_u$ é não determinístico, podemos deixá-lo escolher qual dos dois autômatos ele vai querer simular

em uma dada computação. Portanto, dada uma palavra w a $\mathcal{M}_u$, ele executa a seguinte estratégia:

- escolhe se vai simular $\mathcal{M}$ ou $\mathcal{M}'$;
- executa a simulação escolhida e aceita w apenas se for aceita pelo autômato cuja simulação está sendo executada.

Uma maneira de realizar isto em um autômato finito é criar um novo estado inicial q_0 cuja única função é realizar a escolha de qual dos dois autômatos será simulado. Mais uma vez, como $\mathcal{M}_u$ não é determinístico, podemos deixá-lo decidir, por si próprio, qual o autômato que será simulado.

Ainda assim, resta um problema. De fato, todos os símbolos de w serão necessários para que as simulações de $\mathcal{M}$ e $\mathcal{M}'$ funcionem corretamente. Contudo, nossos autômato precisam consumir símbolos para se mover. Digamos, por exemplo, que ao receber w no estado q_0 o autômato $\mathcal{M}_u$ escolhe simular $\mathcal{M}$ com entrada w . Entretanto, para poder simular $\mathcal{M}$ em w , $\mathcal{M}_u$ precisa deixar q_0 e chegar ao estado inicial q_1 de $\mathcal{M}$ ainda tendo w por entrada, o que não é possível. Resolvemos este problema fazendo com que q_0 comporte-se, simultaneamente, como q_1 ou como q'_1 , dependendo de qual autômato $\mathcal{M}_u$ escolheu simular. Mais precisamente, se $\mathcal{M}_u$ escolheu simular $\mathcal{M}$, então q_0 realiza a transição a partir do primeiro símbolo de w como se fosse uma transição de $\mathcal{M}$ a partir de q_1 por este mesmo símbolo.

Para poder formular isto formalmente, digamos que $w = \sigma u$, onde $\sigma \in \Sigma$ e $u \in \Sigma^*$. Neste caso, se $\mathcal{M}_u$ escolheu simular $\mathcal{M}$, devemos ter que

$$(q_0, w) \vdash (p, u) \text{ se, e somente se, } p \in \delta(q_1, \sigma).$$

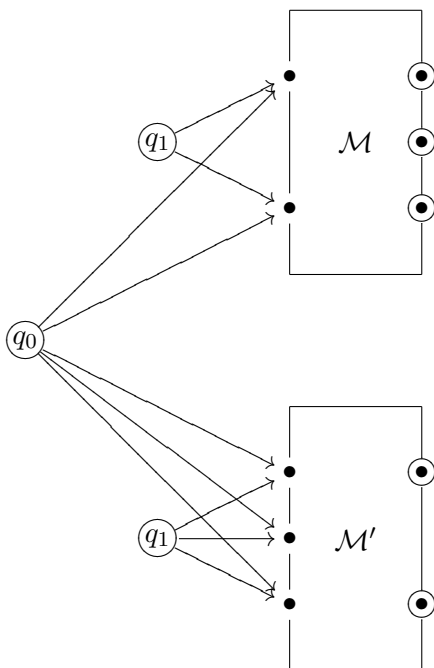
Podemos dissecar o comportamento de $\mathcal{M}_u$ a partir de q_0 como duas escolhas sucessivas:

- (1) primeiro $\mathcal{M}_u$ decide se vai simular $\mathcal{M}$ ou $\mathcal{M}'$;
- (2) a seguir, $\mathcal{M}_u$ escolhe qual a transição que vai ser executada por σ , a partir de q_1 ou q'_1 —dependendo da escolha feita em (1).

Denotando por δ_u a função de transição de $\mathcal{M}_u$, podemos resumir isto escrevendo

$$\delta_u(q_0, \sigma) = \delta(q_1, \sigma) \cup \delta'(q'_1, \sigma).$$

O comportamento geral de $\mathcal{M}_u$ pode ser ilustrado em uma figura, como segue.



Como a figura sugere, a parte q_0 , os estados de $\mathcal{M}_u$ são os mesmos de $\mathcal{M}$ e $\mathcal{M}'$. Quanto aos estados finais, precisamos ser mais cuidadosos. Como, uma vez tendo passado por q_0 , o autômato $\mathcal{M}_u$ meramente simula $\mathcal{M}$ ou $\mathcal{M}'$, só pode haver algum problema se um dos estados q_1 ou q'_1 for final. Por exemplo, se q_1 for final, então $\mathcal{M}$ e, portanto, $\mathcal{M}_u$, deverá aceitar ϵ . Mas isto só pode ocorrer se q_0 for final. Portanto, q_0 deverá ser declarado como final sempre que q_1 ou q'_1 forem finais.

Vamos formalizar a construção de $\mathcal{M}_u$ antes de fazer um exemplo. Listando os vários elementos de $\mathcal{M}_u$ um a um temos

Alfabeto: Σ ;

Estados: $\{q_0\} \cup Q \cup Q'$;

Estado inicial: q_0 ;

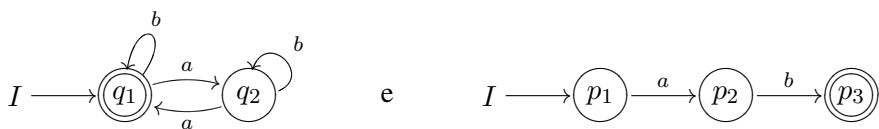
Estados finais:

$$\begin{aligned} &\{q_0\} \cup F \cup F' \text{ se } q_1 \in F \text{ ou } q'_1 \in F' \\ &F \cup F' \text{ se } q_1 \notin F \text{ e } q'_1 \notin F' \end{aligned}$$

Função de transição:

$$\delta_u(q, \sigma) = \begin{cases} \delta(q_1, \sigma) \cup \delta'(q'_1, \sigma) & \text{se } q = q_0 \\ \delta(q, \sigma) & \text{se } q \in Q \\ \delta'(q, \sigma) & \text{se } q \in Q' \end{cases}$$

Vejamos como a construção se comporta em um exemplo. Considere os autômatos finitos $\mathcal{M}$ e $\mathcal{M}'$ cujos grafos são, respectivamente



Gramáticas lineares à direita

Neste capítulo começamos o estudo das gramáticas formais, que serão introduzidas a partir da noção de autômato finito.

1. Exercícios

- Determine gramáticas lineares à direita que gerem as linguagens denotadas pelas seguintes expressões regulares:
 - $(0^* \cdot 1) \cup 0$;
 - $(0^* \cdot 1) \cup (1^* \cdot 0)$;
 - $((0^* \cdot 1) \cup (1^* \cdot 0))^*$.
- Ache a expressão regular que denota a linguagem regular gerada pela gramática com variáveis S, A, B , terminais a, b , símbolo inicial S e regras:

$$S \rightarrow bA \quad | \quad aB \quad | \quad \epsilon$$

$$A \rightarrow abaS$$

$$B \rightarrow babS.$$

- Ache uma gramática linear à direita que gere a seguinte linguagem

$$\{w \in \{0, 1\}^* : w \text{ não contém a seqüência } aa\}.$$
- Ache gramáticas lineares à direita que gerem cada uma das linguagens regulares do exercício 2 da 2ª lista de exercícios.
- Ache autômatos finitos que aceitem as linguagens geradas pelas gramáticas lineares à direita no alfabeto $\{0, 1\}$ e símbolo inicial S , com as seguintes regras:
 - $S \rightarrow 011X, S \rightarrow 11S, X \rightarrow 101Y, Y \rightarrow 111.$
 - $S \rightarrow 0X, X \rightarrow 1101Y, X \rightarrow 11X, Y \rightarrow 11Y, X \rightarrow 1.$

6. Seja L uma linguagem regular no alfabeto Σ . Mostre que $L \setminus \{\epsilon\}$ pode ser gerada por uma gramática linear à direita cujas regras são dos tipos

- $X \rightarrow \sigma Y$ ou
- $X \rightarrow \sigma$,

onde X, Y são variáveis e σ é um símbolo de Σ .

CAPÍTULO 9

Linguagens livres de contexto

Neste capítulo começamos a estudar uma classe de gramáticas fundamental na descrição das linguagens de programação: as gramáticas livres de contexto.¹

1. Gramáticas e linguagens livres de contexto

Já vimos no capítulo anterior que uma gramática é descrita pelos seguintes ingredientes: terminais, variáveis, símbolo inicial e regras. É claro que, em última análise, quando pensamos em uma gramática o que nos vem a cabeça são suas regras; os outros ingredientes são, de certo modo, circunstanciais. Assim, o que diferencia uma classe de gramáticas da outra é o tipo de regras que nos é permitido escrever. No caso de gramáticas lineares à direita, as regras são extremamente rígidas: uma variável só pode ser levada na concatenação de uma palavra nos terminais com alguma variável, além disso a variável tem que estar à direita dos terminais.

Já as linguagens livres de contexto, que estaremos estudando neste capítulo, admitem regras muito mais flexíveis. De fato a única restrição é que à esquerda da seta só pode aparecer uma variável. Talvez você esteja se perguntando: mas o que mais poderia aparecer à esquerda de

¹Agradeço ao David Boechat pelas correções a uma versão anterior deste capítulo

uma seta? Voltaremos a esta questão na seção 2, depois de considerar vários exemplos de linguagens que *são* livres de contexto. Mas antes dos exemplos precisamos dar uma definição formal do que é uma gramática livre de contexto.

Seja $\mathcal{G}$ uma gramática com conjunto de terminais $\mathcal{T}$, conjunto de variáveis $\mathcal{V}$ e símbolo inicial $S \in \mathcal{V}$. Dizemos que $\mathcal{G}$ é *livre de contexto* se todas as suas regras são do tipo $X \rightarrow w$, onde $X \in \mathcal{V}$ e $w \in (\mathcal{T} \cup \mathcal{V})^*$.

Observe que as regras de uma gramática linear à direita são deste tipo. Portanto, toda gramática linear à direita é livre de contexto. Por outro lado, é evidente que nem toda gramática livre de contexto é linear à direita. Um exemplo simples é dado pela gramática $\mathcal{G}_1$ com terminais $T = \{0, 1\}$, variáveis $V = \{S\}$, símbolo inicial S e conjunto de regras

$$\{S \rightarrow 0S1, S \rightarrow \epsilon\}.$$

Como do lado esquerdo de cada seta só há uma variável, $\mathcal{G}_1$ é livre de contexto. Contudo, $\mathcal{G}_1$ não é linear à direita porque $S \rightarrow 0S1$ tem um terminal à direita de uma variável.

Outro exemplo simples é a gramática $\mathcal{G}_2$ com terminais $T = \{0, 1\}$, variáveis $V = \{S, X\}$, símbolo inicial S e conjunto de regras

$$\{S \rightarrow X1X, X \rightarrow 0X, X \rightarrow \epsilon\}.$$

Mais uma vez, apesar de ser claramente livre de contexto, esta gramática não é linear à direita, por causa da regra $S \rightarrow X1X$.

Os dois exemplos construídos acima estão relacionados a linguagens que são velhas conhecidas nossas. Entretanto, para constatar isto precisamos entender como é possível construir uma linguagem a partir de uma gramática livre de contexto. Isto se faz de maneira análoga ao que já ocorria com linguagens lineares à direita.

Seja $\mathcal{G}$ uma gramática livre de contexto com terminais $\mathcal{T}$, variáveis $\mathcal{V}$, símbolo inicial S e conjunto $\mathcal{R}$ de regras, e sejam $w, w' \in (\mathcal{T} \cup \mathcal{V})^*$. Dizemos que w' *pode ser derivada em um passo* a partir de w em $\mathcal{G}$, e escrevemos $w \Rightarrow_{\mathcal{G}} w'$, se w' pode ser obtida a partir de w substituindo-se uma variável que aparece em w pelo lado direito de alguma regra da gramática $\mathcal{G}$. Em outras palavras, para que $w \Rightarrow_{\mathcal{G}} w'$ é preciso que:

- exista uma decomposição da forma $w = uXv$, onde $u, v \in (\mathcal{T} \cup \mathcal{V})^*$ e $X \in \mathcal{V}$, e
- exista uma regra $X \rightarrow \alpha$ em $\mathcal{R}$ tal que $w' = u\alpha v$.

Como já estamos acostumados a fazer, dispensaremos o $\mathcal{G}$ subscrito na notação acima quando não houver dúvidas quanto à gramática que está sendo considerada (isto é, quase sempre!).

Como já ocorria com as gramáticas lineares à direita, estaremos gerando palavras a partir de uma gramática livre de contexto pelo encadeamento de várias derivações de um passo. Assim, dizer que w' pode ser *derivada a partir de w em $\mathcal{G}$ em n passos* significa que existem palavras $w_1, \dots, w_{n-1} \in (\mathcal{T} \cup \mathcal{V})^*$ tais que

$$w = w_0 \Rightarrow w_1 \Rightarrow \dots \Rightarrow w_{n-1} \Rightarrow w_n = w'$$

Chamamos a isto uma *derivação* de w' a partir de w em $\mathcal{G}$ e escrevemos $w \Rightarrow^n w'$. Caso não seja conveniente indicar o número de passos da derivação substituímos o n por uma $*$ como já estamos acostumados a fazer. É conveniente também adotar a convenção de que w pode ser derivada dela própria em zero passos, de modo que faz sentido escrever $w \Rightarrow^* w$.

O conjunto de todas as palavras de $\mathcal{T}^*$ que podem ser derivadas a partir do símbolo inicial S na gramática $\mathcal{G}$ é a *linguagem gerada por $\mathcal{G}$* . Denotando $L(\mathcal{G})$ a linguagem gerada por $\mathcal{G}$, e usando a notação definida acima, temos que

$$L(\mathcal{G}) = \{w \in \mathcal{T}^* : \text{existe uma derivação } S \Rightarrow^* w \text{ em } \mathcal{G}\}.$$

Vejamos alguns exemplos. Seja $\mathcal{G}_1$ a gramática definida acima. Temos que

$$S \Rightarrow 0S1 \Rightarrow 0^2S1^2 \Rightarrow 0^21^2.$$

Note que $S \Rightarrow 0S1$ e $0S1 \Rightarrow 0^2S1^2$ são derivações de um passo em $\mathcal{G}_1$ porque, em ambos os casos a palavra à direita de $\Rightarrow$ foi obtida da que está à esquerda trocando-se S por $0S1$. Isto é permitido porque $S \rightarrow 0S1$ é uma regra de $\mathcal{G}_1$. Já $0^2S1^2 \Rightarrow 0^21^2$ foi obtida trocando-se S por ϵ . Podemos fazer isto por que $S \rightarrow \epsilon$ também é uma regra de $\mathcal{G}_1$. Toda esta cadeia de derivações pode ser abreviada como

$$S \Rightarrow^3 0^21^1 \text{ ou } S \Rightarrow^* 0^21^1.$$

Por outro lado, $0^2S1^2 \Rightarrow 0^21^3$ *não* é uma derivação legítima em $\mathcal{G}_1$ porque neste caso a regra usada foi $S \rightarrow 1$, que não pertence a $\mathcal{G}_1$.

Do que fizemos acima segue que $0^21^2 \in L(\mathcal{G}_1)$. Mas podemos ser muito mais ambiciosos e tentar determinar todas as palavras de $L(\mathcal{G}_1)$. Para começar, note que se $S \Rightarrow^n w$ em $\mathcal{G}_1$ e $w \notin \mathcal{T}^*$, então $w = 0^n S 1^n$. Para provar isto basta usar indução em n . Por outro lado, como a única regra de $\mathcal{G}_1$ que permite eliminar a variável S é $S \rightarrow \epsilon$, concluímos que toda palavra derivada a partir de S em $\mathcal{G}_1$ é da forma $0^n 1^n$, para algum inteiro $n \geq 0$. Assim,

$$L(\mathcal{G}_1) = \{0^n 1^n : n \geq 0\}.$$

Já vimos no capítulo 4, que esta linguagem não é regular. Em particular, não existe nenhuma gramática linear à direita que gere $L(\mathcal{G}_1)$.

Passando à gramática $\mathcal{G}_2$, temos a derivação

$$S \Rightarrow \underline{X}1X \Rightarrow 0\underline{X}1X \Rightarrow 0^2X1\underline{X} \Rightarrow 0^2X1 \Rightarrow 0^21.$$

Note que, na derivação acima, algumas ocorrências da variável X foram sublinhadas. Fizemos isto para indicar à qual instância da variável X foi aplicada a regra da gramática que leva ao passo seguinte da derivação. Assim, no segundo passo da derivação, aplicamos a regra $X \rightarrow 0X$ ao X mais à esquerda $X1X$. Com isto este X foi trocado por $0X$, mas nada foi feito ao segundo X . Coisa semelhante ocorreu no passo seguinte. Além disso, a regra $X \rightarrow 0X$ nunca foi aplicada ao segundo X . Fica claro deste exemplo que:

A cada passo de uma derivação apenas uma ocorrência de uma variável pode ser substituída pelo lado direito de uma seta. Além disso, cada ocorrência de uma variável é tratada independentemente da outra.

Sempre que for conveniente, sublinharemos a instância da variável à qual a regra está sendo aplicada em um determinado passo da derivação.

Vamos tentar, também neste caso, determinar todas as palavras que podem ser derivadas a partir de S em $\mathcal{G}_2$. Suponhamos que $S \Rightarrow^* w$ em $\mathcal{G}_2$. É fácil ver que em w

- há um único 1;
- pode haver, no máximo, duas ocorrências de X , uma de cada lado do 1.

Assim, w pode ser de uma das seguintes formas

$$0^n X 1 0^m X \text{ ou } 0^n 1 0^m X \text{ ou } 0^n X 1 0^m \text{ ou } 0^n 1 0^m$$

Portanto, se $w \in \mathcal{T}^*$ e $S \Rightarrow^* w$, então $w = 0^n 1 0^m$, para inteiros $m, n \geq 0$. Concluimos que

$$L(\mathcal{G}_2) = \{0^n 1 0^m : n, m \geq 0\}.$$

Mas esta é, na verdade, uma linguagem regular, que pode ser descrita na forma $L(\mathcal{G}_2) = 0^* 1 0^*$. Temos assim um exemplo de gramática livre de contexto que não é linear à direita mas que gera uma linguagem regular.

Voltando ao caso geral, seja L uma linguagem no alfabeto Σ . Dizemos que L é *livre de contexto* se existe pelo menos uma gramática livre de contexto $\mathcal{G}$, com conjunto de terminais Σ , tal que $L(\mathcal{G}) = L$.

Portanto, toda linguagem regular é livre de contexto. Isto ocorre porque, pelo teorema de Kleene, uma linguagem regular sempre pode ser gerada por uma gramática linear à direita. Mas, como já vimos, toda gramática linear à direita é livre de contexto.

Por outro lado, nem toda linguagem livre de contexto é regular. Por exemplo, vimos na seção 3 do capítulo 4 que a linguagem $\{0^n 1^n : n \geq 0\}$ não é regular. No entanto, ela é gerada pela gramática $\mathcal{G}_1$ e, portanto, é livre de contexto. Na seção 3 veremos mais exemplos de linguagens livres de contexto, incluindo várias que não são regulares.

Antes que você comece a alimentar falsas esperanças, é bom esclarecer que existem linguagens que não são livres de contexto. Para mostrar, *diretamente da definição*, que uma linguagem L não é livre de contexto seria necessário provar que não existe nenhuma gramática livre de contexto que gere L , o que não parece possível. Como no caso de linguagens regulares, a estratégia mais prática consiste em mostrar que há uma propriedade de toda linguagem livre de contexto que não é satisfeita por L . Veremos como fazer isto no capítulo 11.

2. Linguagens sensíveis ao contexto

É hora de voltar à questão de como seria uma gramática que não é livre de contexto.

Lembre-se que o que caracteriza as gramáticas livres de contexto é o fato de que todas as suas regras têm apenas uma variável (e nenhum terminal) do lado esquerdo da seta. Na prática isto significa que sempre que X aparece em uma palavra, ele pode ser trocado por w , desde que $X \rightarrow w$ seja uma regra da gramática.

Para uma gramática não ser livre de contexto basta que, do lado esquerdo da seta de alguma de suas regras, apareça algo mais complicado que uma variável isolada. Um exemplo simples, mas importante, é a gramática com terminais $\{a, b, c\}$, variáveis $\{S, X, Y\}$, símbolo inicial S e regras

$$\begin{aligned} S &\rightarrow abc \\ S &\rightarrow aXbc \\ Xb &\rightarrow bX \\ Xc &\rightarrow Ybc^2 \\ bY &\rightarrow Yb \\ aY &\rightarrow a^2 \\ aY &\rightarrow a^2X. \end{aligned}$$

Considere uma derivação nesta gramática que comece com $S \Rightarrow aXbc$. Queremos, no próximo passo, substituir o X por alguma coisa. Mas, apesar de haver duas regras com X à esquerda da seta na gramática, só uma delas pode ser aplicada. Isto ocorre porque na palavra $aXbc$ o X

vem seguido de b , e a única regra em que X aparece neste ‘contexto’ é $Xb \rightarrow bX$. Aplicando esta regra, obtemos

$$S \Rightarrow aXbc \Rightarrow abXc \Rightarrow abYbc^2.$$

Continuando desta forma, podemos derivar a palavra $a^2b^2c^2$ nesta gramática. A derivação completa é a seguinte:

$$S \Rightarrow aXbc \Rightarrow abXc \Rightarrow abYbc^2 \Rightarrow aYb^2c^2 \Rightarrow a^2b^2c^2.$$

De maneira semelhante, podemos derivar qualquer palavra da forma $a^n b^n c^n$, com $n \geq 1$. De fato, pode-se mostrar que o conjunto das palavras desta forma constitui toda a linguagem gerada pela gramática dada acima.

Este exemplo se encaixa em uma classe de gramáticas chamadas de sensíveis ao contexto. A definição formal é a seguinte. Seja $\mathcal{G}$ uma gramática com conjunto de terminais T , conjunto de variáveis V e símbolo inicial S . Dizemos que $\mathcal{G}$ é *sensível ao contexto* se todas as regras de $\mathcal{G}$ são da forma $u \rightarrow v$, onde $u, v \in (\mathcal{T} \cup V)^*$ e $|u| \leq |v|$. Esta última condição é claramente satisfeita pelas regras de uma gramática livre de contexto, de modo que toda gramática livre de contexto é sensível ao contexto

Diremos que uma linguagem L no alfabeto Σ é *sensível ao contexto* se existe uma gramática sensível ao contexto, com conjunto de terminais Σ , que gera L . Portanto, toda linguagem livre de contexto é sensível ao contexto

A discussão acima mostra que a linguagem

$$\{a^n b^n c^n : n \geq 1\}$$

é sensível ao contexto. Entretanto, como provaremos no capítulo 11 não existe nenhuma gramática livre de contexto que gere esta linguagem. Temos, assim, que a classe das linguagens livres de contexto está propriamente contida na classe das linguagens sensíveis ao contexto. A relação entre os diversos tipos de linguagens é detalhada na hierarquia de Chomsky, que discutiremos em um capítulo posterior.

3. Mais exemplos

Nesta seção veremos mais exemplos de linguagens livres de contexto. Começamos com uma gramática que gera fórmulas que sejam expressões aritméticas envolvendo apenas os operadores soma e multiplicação, além dos parênteses. Isto é, expressões da forma

$$(3.1) \quad ((x + y) * x) * (z + w) * y),$$

onde x, y, z e w são variáveis (no sentido em que o termo é usado em álgebra).

Note que, para saber se uma dada expressão é legítima, não precisamos conhecer as variáveis que nela aparecem, mas apenas sua localização em relação aos operadores. Assim, podemos construir uma gramática mais simples, em que as posições ocupadas por variáveis em uma expressão são todas marcadas com um mesmo símbolo. Este símbolo é chamado de *identificador*, e abreviado *id*. Portanto, o identificador *id* será um terminal da gramática que vamos construir, juntamente com os operadores $+$ e $*$ e os parênteses $($ e $)$. Substituindo as variáveis da expressão (3.1) pelo identificador, obtemos

$$(3.2) \quad ((id + id) * id) * (id + id) * id).$$

Em resumo, queremos construir uma gramática livre de contexto $\mathcal{G}_{exp}$, com conjunto terminais $\{+, *, (,), id\}$, que gere todas as expressões aritméticas legítimas nos operadores $+$ e $*$. Note que $\mathcal{G}_{exp}$ deve gerar todas as expressões legítimas, e apenas estas. Isto é, queremos gerar expressões como (3.2), mas não como

$$(+id)id * .$$

Na prática estas expressões aritméticas são produzidas recursivamente, somando ou multiplicando expressões mais simples, sem esquecer de pôr os parênteses no devido lugar. Para obter a gramática, podemos criar uma variável E (de *expressão*), e introduzir as seguintes regras para combinação de expressões:

$$\begin{aligned} E &\rightarrow E + E \\ E &\rightarrow E * E \\ E &\rightarrow (E). \end{aligned}$$

Finalmente, para eliminar E e fazer surgir o identificador, adicionamos a regra $E \rightarrow id$.

Vejamos como derivar a expressão $id + id * id$ nesta gramática. Para deixar claro o que está sendo feito em cada passo, vamos utilizar a convenção, estabelecida na seção em 1, de sublinhar a instância da variável à qual uma regra está sendo aplicada:

$$(3.3) \quad \underline{E} \Rightarrow E + \underline{E} \Rightarrow \underline{E} + E * \underline{E} \Rightarrow id + \underline{E} * E \Rightarrow id + id * \underline{E} \Rightarrow id + id * id.$$

Construímos esta derivação aplicando em cada passo uma regra da gramática a uma variável escolhida sem nenhum critério especial. Entretanto podemos ser mais sistemáticos. Por exemplo, poderíamos, em

cada passo da derivação, aplicar uma regra sempre à variável que está mais à esquerda da palavra. Usando esta estratégia chegamos a uma derivação de $id + id * id$ diferente da que foi obtida acima:

$$E \Rightarrow E + E \Rightarrow id + E \Rightarrow id + E * E \Rightarrow id + id * E \Rightarrow id + id * id.$$

Neste caso não há necessidade de sublinhar a variável à qual a regra está sendo aplicada já que, em cada passo, escolhemos sempre a que está mais à esquerda da palavra.

As derivações deste tipo são muito importantes no desenvolvimento da teoria e em suas aplicações. Por isso é conveniente ter um nome especial para elas. Seja $\mathcal{G}$ uma gramática livre de contexto e $w \in L(\mathcal{G})$. Uma *derivação mais à esquerda* de w em $\mathcal{G}$ é aquela na qual, em cada passo, a variável à qual a regra é aplicada é a que está mais à esquerda da palavra. Analogamente, podemos definir *derivação mais à direita* em uma gramática livre de contexto.

Uma gramática como $\mathcal{G}_{exp}$ é usada em uma linguagem de programação com duas finalidades diferentes. Em primeiro lugar, para verificar se as expressões aritméticas de um programa estão bem construídas; em segundo, para informar ao computador qual é a interpretação correta destas expressões. Por exemplo, o computador tem que ser informado sobre a precedência correta entre os operadores soma e multiplicação. Isto é necessário para que, na ausência de parêntesis, o computador saiba que deve efetuar primeiro as multiplicações e só depois as somas. Do contrário, confrontado com $id + id * id$ ele não saberia como efetuar o cálculo da forma correta. Voltaremos a esta questão em mais detalhes no próximo capítulo.

Analizando $\mathcal{G}_{exp}$ com cuidado, verificamos que permite derivar (id) . Apesar do uso dos parênteses ser desnecessário neste caso, não se trata de uma expressão aritmética ilegítima. Entretanto, não é difícil resolver este problema introduzindo uma nova variável, como é sugerido no exercício 5.

O segundo exemplo que desejamos considerar é o de uma gramática que gere a linguagem formada pelos palíndromos no alfabeto $\{0, 1\}$. Lembre-se que um palíndromo é uma palavra cujo reflexo é igual a ela própria. Isto é, w é um palíndromo se e somente se $w^R = w$. Precisamos de dois fatos sobre palíndromos que são consequência imediata da definição. Digamos que $w \in (0 \cup 1)^*$ e que σ é 0 ou 1; então:

- (1) w é palíndromo se e somente se w começa e termina com o mesmo símbolo;
- (2) $\sigma w \sigma$ é palíndromo se e somente se w também é palíndromo.

Isto sugere que os palíndromos podem ser construídos recursivamente onde, a cada passo, ladeamos um palíndromo já construído por duas letras iguais.

Com isto podemos passar à construção da gramática. Vamos chamá-la de $\mathcal{G}_{pal}$: terá terminais $\{0, 1\}$ e apenas uma variável S , que fará o papel de símbolo inicial. As observações acima sugerem as seguintes regras

$$S \rightarrow 0S0$$

$$S \rightarrow 1S1$$

Estas regras ainda não bastam, porque apenas com elas não é possível eliminar S da palavra. Para fazer isto precisamos de, pelo menos, mais uma regra. A primeira idéia seria introduzir $S \rightarrow \epsilon$. Entretanto, se fizermos isto só estaremos gerando palíndromos que não têm um símbolo no meio; isto é, os que têm comprimento par. Para gerar os de comprimento ímpar é necessário também introduzir regras que permitam substituir S por um terminal; isto é,

$$S \rightarrow 0 \text{ e } S \rightarrow 1.$$

Podemos resumir o que fizemos usando a seguinte notação. Suponha que $\mathcal{G}$ é uma gramática livre de contexto que tem várias regras

$$X \rightarrow w_1, \dots, X \rightarrow w_k$$

todas com uma mesma variável do lado esquerdo. Neste caso escrevemos abreviadamente

$$X \rightarrow w_1 \mid w_2 \dots \mid w_k,$$

onde a barra vertical tem o valor de ‘ou’. Como $\mathcal{G}_{pal}$ tem uma única variável, podemos escrever todas as suas regras na forma

$$S \rightarrow 0S0 \mid 1S1 \mid \epsilon \mid 0 \mid 1.$$

De quebra, obtivemos a gramática $\mathcal{G}_{pal}^+$ que gera os palíndromos de comprimento par. A única diferença entre as duas gramáticas é que o conjunto de regras de $\mathcal{G}_{pal}^+$ é ainda mais simples:

$$S \rightarrow 0S0 \mid 1S1 \mid \epsilon.$$

4. Combinando gramáticas

Muitas vezes é possível decompor uma linguagem livre de contexto L como união, concatenação ou estrela de outras linguagens livres de contexto. Nesta seção introduzimos técnicas que facilitam a construção

de uma gramática livre de contexto para L a partir das gramáticas das suas componentes.

Começaremos com um exemplo cuja verdadeira natureza só será revelada no próximo capítulo. Considere a linguagem

$$L_{in} = \{a^i b^j c^k : i, j, k \geq 0 \text{ e } i = j \text{ ou } j = k\}.$$

Podemos decompô-la, facilmente, como união de outras duas, a saber

$$L_1 = \{a^i b^j c^k : i, j, k \geq 0 \text{ e } i = j\}, \quad \text{e}$$

$$L_2 = \{a^i b^j c^k : i, j, k \geq 0 \text{ e } j = k\}.$$

Além disso, $L_1 = L_3 \cdot c^*$ e $L_2 = a^* \cdot L_4$, onde

$$L_3 = \{a^i b^j : i, j \geq 0 \text{ e } i = j\}, \quad \text{e} \quad L_4 = \{b^j c^k : j, k \geq 0 \text{ e } j = k\}.$$

Chegados a este ponto, já conseguimos obter uma decomposição de L_{in} em linguagens para as quais conhecemos gramáticas. De fato, a^* e c^* são linguagens regulares para as quais existem gramáticas lineares à direita bastante simples. Por outro lado, a gramática $\mathcal{G}_1$ definida na seção 1 gera uma linguagem análoga a L_3 e L_4 . Resta-nos descobrir como esta decomposição pode ser usada para construir uma gramática para L_{in} . Contudo, é preferível discutir o problema de combinar gramáticas em geral, e só depois aplicá-lo no exemplo acima.

Suponhamos, então, que $\mathcal{G}_1$ e $\mathcal{G}_2$ são gramáticas livres de contexto que geram linguagens L_1 e L_2 , respectivamente. O que queremos são receitas que nos permitam obter gramáticas livres de contexto que gerem $L_1 \cup L_2$ e $L_1 \cdot L_2$ a partir de $\mathcal{G}_1$ e $\mathcal{G}_2$.

Digamos que $\mathcal{G}_1$ tem ingredientes $(\mathcal{T}, \mathcal{V}_1, \mathcal{S}_1, \mathcal{R}_1)$ e $\mathcal{G}_2$ ingredientes $(\mathcal{T}, \mathcal{V}_2, \mathcal{S}_2, \mathcal{R}_2)$. Observe que estamos supondo que as duas gramáticas têm os mesmos terminais, e que os conjuntos de variáveis são disjuntos; isto é, $\mathcal{V}_1 \cap \mathcal{V}_2 = \emptyset$.

Vamos começar com $L_1 \cup L_2$. Neste caso a nova gramática, que chamaremos de $\mathcal{G}_\cup$, deve gerar uma palavra que pertença a L_1 ou a L_2 . Assim, o conjunto de regras de $\mathcal{G}_\cup$ precisa conter $\mathcal{R}_1$ e $\mathcal{R}_2$. Mas queremos também que, depois do primeiro passo, a derivação só possa proceder usando regras de uma das duas gramáticas. Fazemos isto criando um novo símbolo inicial S e duas novas regras

$$S \rightarrow S_1 \text{ e } S \rightarrow S_2.$$

Assim, o primeiro passo da derivação força uma escolha entre S_1 e S_2 . Esta escolha determina de maneira inequívoca se a palavra a ser gerada estará em L_1 ou L_2 . Em outras palavras, depois do primeiro passo a derivação fica obrigatoriamente restrita às regras de uma das duas gramáticas. Mais precisamente, $\mathcal{G}_\cup$ fica definida pelos seguintes ingredientes:

Terminais: $\mathcal{T}_1 \cup \mathcal{T}_2$;
Variáveis: $\mathcal{V}_1 \cup \mathcal{V}_2 \cup \{S\}$;
Símbolo inicial: S ;
Regras: $\mathcal{R}_1 \cup \mathcal{R}_2 \cup \{S \rightarrow S_1, S \rightarrow S_2\}$.

Podemos proceder de maneira semelhante para criar uma gramática $\mathcal{G}_\bullet$ que gere a concatenação $L_1 \cdot L_2$. Neste caso, as palavras que queremos derivar são construídas escrevendo uma palavra de L_1 seguida de uma palavra de L_2 . Como as palavras de L_1 são geradas a partir de S_1 e as de L_2 a partir de S_2 , basta acrescentar $S \rightarrow S_1 S_2$ às regras de $\mathcal{G}_1$ e $\mathcal{G}_2$. Os ingredientes da gramática $\mathcal{G}_\bullet$ são:

Terminais: $\mathcal{T}_1 \cup \mathcal{T}_2$;
Variáveis: $\mathcal{V}_1 \cup \mathcal{V}_2 \cup \{S\}$;
Símbolo inicial: S ;
Regras: $\mathcal{R}_1 \cup \mathcal{R}_2 \cup \{S \rightarrow S_1 S_2\}$.

Finalmente, há uma receita semelhante às que foram dadas acima para criar uma gramática livre de contexto para L^* a partir de uma gramática livre de contexto que gere L . Deixamos os detalhes desta construção para o exercício 7.

Podemos, agora, voltar à linguagem L_{in} . Como $L_{in} = L_3 \cdot c^* \cup a^* \cdot L_4$, precisamos começar criando gramáticas que gerem as linguagens L_3, L_4, c^* e a^* . Para L_3 e L_4 podemos usar adaptações da gramática $\mathcal{G}_1$ da seção 1. Já c^* e a^* são regulares, geradas por gramáticas lineares à direita extremamente simples. Resumimos os ingredientes destas várias gramáticas na tabela abaixo:

Linguagem	L_3	L_4	a^*	c^*
Terminais	$\{0, 1\}$	$\{0, 1\}$	$\{0, 1\}$	$\{0, 1\}$
Variáveis	S_1	S_2	S_3	S_4
Símbolo inicial	S_1	S_2	S_3	S_4
Regras	$S_1 \rightarrow aS_1b$ $S_1 \rightarrow \epsilon$	$S_2 \rightarrow bS_2c$ $S_2 \rightarrow \epsilon$	$S_3 \rightarrow aS_3$ $S_3 \rightarrow \epsilon$	$S_4 \rightarrow cS_4$ $S_4 \rightarrow \epsilon$

Seguindo a receita dada acima para a gramática de uma concatenação, obtemos:

Linguagem	$L_3 \cdot c^*$	$a^* \cdot L_4$
Terminais	$\{0, 1\}$	$\{0, 1\}$
Variáveis	$\{S', S_1, S_4\}$	$\{S'', S_2, S_3\}$
Símbolo inicial	S'	S''
Regras	$S' \rightarrow S_1 S_4$ $S_1 \rightarrow a S_1 b$ $S_1 \rightarrow \epsilon$ $S_4 \rightarrow c S_4$ $S_3 \rightarrow \epsilon$	$S'' \rightarrow S_2 S_3$ $S_2 \rightarrow b S_2 c$ $S_2 \rightarrow \epsilon$ $S_3 \rightarrow a S_3$ $S_4 \rightarrow \epsilon$

Resta-nos, apenas, proceder à união destas gramáticas, o que nos dá uma gramática livre de contexto para L_{in} , com os seguintes ingredientes:

Terminais: $\{0, 1\}$;

Variáveis: $S, S', S'', S_1, S_2, S_3, S_4$;

Símbolo inicial: S ;

Regras: $\{S \rightarrow S', S \rightarrow S'', S' \rightarrow S_1 S_4, S'' \rightarrow S_2 S_3, S_1 \rightarrow a S_1 b, S_2 \rightarrow b S_2 c, S_1 \rightarrow \epsilon, S_2 \rightarrow \epsilon, S_4 \rightarrow c S_4, S_3 \rightarrow a S_3, S_3 \rightarrow \epsilon, S_4 \rightarrow \epsilon\}$.

5. Exercícios

1. Considere a gramática $\mathcal{G}$ com variáveis S, A , terminais a, b , símbolo inicial S e regras

$$S \rightarrow AA$$

$$A \rightarrow AAA \mid a \mid bA \mid Ab$$

- (a) Quais palavras de $L(\mathcal{G})$ podem ser produzidas com derivações de até 4 passos?
 - (b) Dê pelo menos 4 derivações distintas da palavra $babbab$.
 - (c) Para $m, n, p > 0$ quaisquer, descreva uma derivação em $\mathcal{G}$ de $b^m ab^n ab^p$.
2. Determine gramáticas livres de contexto que gerem as seguintes linguagens:
 - (a) $\{(01)^i : i \geq 1\}$;
 - (b) $\{1^{2n} : n \geq 1\}$;
 - (c) $\{0^i 1^{2i} : i \geq 1\}$;
 - (d) $\{w \in \{0, 1\}^* : w \text{ em que o número de 0s e 1s é o mesmo}\}$;
 - (e) $\{w c w^r : w \in \{0, 1\}^*\}$;

- (f) $\{w : w = w^r \text{ onde } w \in \{0, 1\}^*\}$.
3. Considere o alfabeto $\Sigma = \{0, 1, (,), \cup, *, \emptyset\}$. Construa uma gramática livre de contexto que gere todas as palavras de Σ^* que são expressões regulares em $\{0, 1\}$.

4. A gramática livre de contexto $\mathcal{G}$ cujas regras são

$$S \rightarrow 0S1 \mid 0S0 \mid 1S0 \mid 1S1 \mid \epsilon$$

não é linear à direita. Entretanto, $L(\mathcal{G})$ é uma linguagem regular. Ache uma gramática linear à direita $\mathcal{G}'$ que gere $L(\mathcal{G})$.

5. Modifique a gramática $\mathcal{G}_{exp}$ (introduzindo novas variáveis) de modo que não seja mais possível derivar a expressão (id) nesta gramática.
6. Seja $\mathcal{G}$ uma gramática livre de contexto com conjunto de terminais T , conjunto de variáveis V e símbolo inicial S . Dizemos que $\mathcal{G}$ é *linear* se todas as suas regras são da forma

$$X \rightarrow \alpha Y \beta \text{ ou } X \rightarrow \alpha,$$

onde X e Y são variáveis e $\alpha, \beta \in T^*$. Isto é pode haver no máximo uma variável do lado direito de cada regra de $\mathcal{G}$. Uma linguagem é *linear* se pode ser gerada por alguma gramática linear. Mostre que se L é linear então L pode ser gerada por uma gramática cujas regras são de um dos 3 tipos seguintes:

$$X \rightarrow \sigma Y \text{ ou } X \rightarrow Y \sigma \text{ ou } X \rightarrow \sigma$$

onde X é uma variável e σ é um terminal ou $\sigma = \epsilon$.

7. Seja $\mathcal{G}$ uma gramática livre de contexto que gere uma linguagem L . Mostre como construir, a partir de $\mathcal{G}$, uma gramática livre de contexto que gera L^* .
SUGESTÃO: $S \rightarrow SS$.

8. Seja L uma linguagem livre de contexto. Mostre que $L^r = \{w^r : w \in L\}$ também é livre de contexto.